

ParkingMAS

Intelligent Systems Engineering Report

Bedeschi Federica

federica.bedeschi4@studio.unibo.it

September 2026

Disclaimer: AI tools were used for assistance in writing this report and developing the project code; all the generated content was reviewed and adapted by the author, who takes full responsibility for the entire final work.

Contents

1	Requirement Specification	2
1.1	Description	2
1.2	Functional Requirements	2
1.3	Non-Functional Requirements	3
1.4	Implementation Requirements	3
2	Design	4
2.1	Architecture	4
2.2	Infrastructure	5
2.3	Modelling	7
2.3.1	Environment Modelling	7
2.3.2	Agent Modelling	8
2.4	Inter-Agent Communication	9
2.4.1	Parking information collection	10
2.4.2	Parking search	11
2.4.3	Parking entry and exit	12
2.4.4	Parking availability updates	14
3	Implementation	16
3.1	Environment	16
3.1.1	GUI	17
3.2	Agents	17
3.2.1	Driver Agent	17
3.2.2	Area Agent	19
3.2.3	Parking Agent	20
4	Testing	21
5	Deployment	22
6	Future works	24

Requirement Specification

1.1 Description

ParkingMAS is a **distributed Multi-Agent System** that models a parking system. Drivers are located in a city divided into areas, each containing several parking lots with limited spots. Drivers can autonomously move through the city and search for, enter, and exit parking lots.

1.2 Functional Requirements

1. Drivers must be able to move through the city;
2. Drivers must be able to search for available parking lots by specifying the minimum number of available spots required, and obtain the closest parking lot matching the request;
3. Drivers must be able to enter a parking lot if a parking spot is available; otherwise, they must receive a negative response;
4. Drivers must be able to exit a parking lot;
5. Drivers must be able to gracefully handle communication or operation failures and use backup plans when necessary;
6. The system must correctly handle concurrent parking requests;
7. The system must provide a GUI for observing the behavior of the simulation.

1.3 Non-Functional Requirements

1. *Concurrency*: the system must support the concurrent execution of multiple agents and operations;
2. *Distribution*: the system must support the distribution of agents across multiple execution nodes;
3. *Fault tolerance*: the system must tolerate communication failures and temporary agent unavailability without causing data loss or an inconsistent system state;
4. *Scalability*: the system must support horizontal scaling without significantly compromising system performance.

1.4 Implementation Requirements

1. *BDI agents*: the system must model autonomous agents according to the Belief-Desire-Intention (BDI) paradigm, using beliefs, goals, and plans to support autonomous reasoning and decision-making;
2. *AgentSpeak and Jason*: agent behavior must be specified using AgentSpeak and implemented and executed through the Jason framework;
3. *JADE*: the system must use the JADE framework to provide the underlying agent infrastructure and support inter-agent communication.

Design

2.1 Architecture

The system architecture, as shown in Figure 2.1, can be divided into two main parts: the **agents** and the **environment**, which follows the **Model-View-Controller (MVC) pattern**. The MVC consists of the **ParkingEnvironment**, which acts as the *Controller*; the **City**, which represents the *Model*; and the **CityView**, which represents the *View*. The **City** encapsulates the state of the environment and the operations that can be performed on it. The **ParkingEnvironment** acts as the *Controller* by mediating between the **City** and the **CityView**. It also mediates the interaction between the environment and the agents by providing them with percepts and allowing them to perform actions that modify the state of the **City**.

The agents are the autonomous components of the MAS, and in addition to interacting with the **ParkingEnvironment**, they interact with each other. The MAS includes three types of agents:

- **Driver Agent**: searches for a suitable parking lot by interacting with area agents, navigates to the selected parking lot, enters it, and eventually exits and returns home;
- **Area Agent**: is responsible for a specific area of the city, maintains information about the parking lots in that area, and provides parking information to driver agents;
- **Parking Agent**: represents a parking lot, handles requests from driver agents to enter or exit, and manages its available spots.

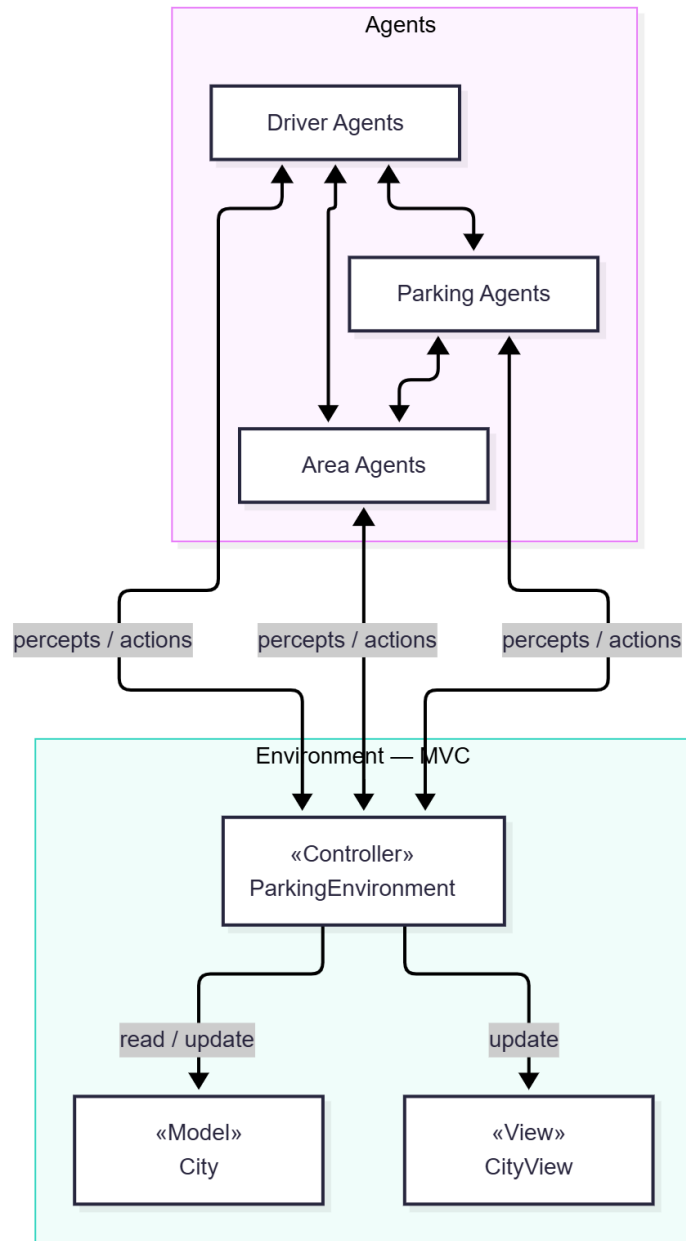


Figure 2.1: System architecture.

2.2 Infrastructure

The system infrastructure is organized into **multiple JADE containers**, each hosting a specific set of agents, as shown in Figure 2.2. Each area of

the city is associated with a separate container hosting its `area_agent` and the `parking_agents` belonging to that area. The `driver_agents` are hosted in a dedicated container. This organization aims to separate the agents according to their **physical distribution**, allowing them to interact through the communication infrastructure as they would in a real-world distributed system.

A **Main Container** hosts the environment, including all the MVC components. The `ParkingEnvironment` provides the interface between the agents and the model, handling agent percepts and actions and therefore acting as the unique access point to the environment. The Main Container therefore represents the **centralized part** of the infrastructure on which the other containers depend, while the agents themselves remain distributed across the different containers.

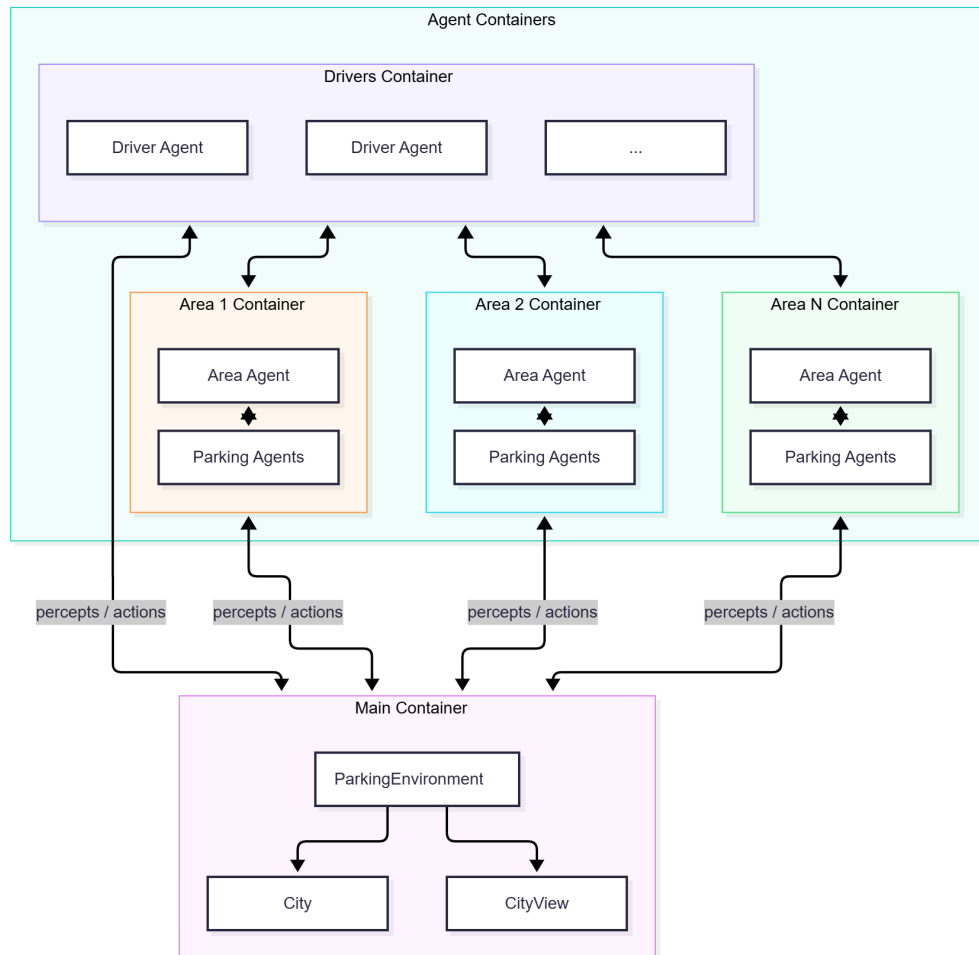


Figure 2.2: System infrastructure.

2.3 Modelling

The ParkingMAS is modelled as a system of autonomous agents interacting with a shared environment. The agents follow the **BDI paradigm** and are implemented using AgentSpeak and Jason. Their behavior is defined through beliefs, rules, goals, and plans, which are selected and executed in response to events and requests. Agents interact with the environment through percepts and actions, and communicate with other agents to gain additional information required for their decision-making.

2.3.1 Environment Modelling

The environment's model is the **City**, which represents a city divided into areas that contain parking lots and in which drivers can move. It maintains the physical state of the city, including the positions of its entities, and provides operations to modify this state, such as drivers' movement. The environment is therefore responsible for maintaining the authoritative state of the physical environment, while agents maintain their own partial knowledge of that state. The city is represented as a two-dimensional space characterized by a width and a height. It contains different areas, parking lots, and drivers. While parking lots have a fixed position, drivers have a dynamic position and are associated with an area based on their current position.

The **ParkingEnvironment** separates the state of the world (the **City**) from the knowledge maintained by individual agents. Agents do not directly access the **City** state. Instead, the **ParkingEnvironment** acts as an interface between the agents and the environment by providing percepts and executing the actions requested by the agents.

Percepts

Each agent receives a different set of percepts depending on its type:

- Driver agents receive their current position, current area, and nearest area (if present);
- Parking agents receive their position;
- Area agents do not receive percepts from the environment, as their initial beliefs and the information acquired through inter-agent communication are sufficient to autonomously carry out their behaviors.

Actions

Agents can affect the environment through a set of actions exposed by the `ParkingEnvironment`. Three main actions are available:

- **Movement**: moves a driver one step in a specified direction;
- **Parking entry**: makes a driver enter a parking lot;
- **Parking exit**: makes a driver exit a parking lot in a specified direction.

The `ParkingEnvironment` translates these actions into operations on the `City`. The result of each operation is returned to the agent, allowing the agent to react to successful or unsuccessful actions.

2.3.2 Agent Modelling

The agents are modelled according to their roles within the parking system, with each agent maintaining its own knowledge and behavior. Although all agents follow the same BDI-based model, their goals and plans differ according to their responsibilities.

Driver Agent

The driver agent represents a driver moving through the city and is responsible for autonomously finding and using a suitable parking lot. Its behavior is primarily goal-driven, with plans allowing it to progressively achieve its parking-related goals. The driver agent's main **beliefs** include information about its current position and area, the nearest area, its home location, and parking availability requirements. As previously mentioned, some of this information is obtained through environment percepts. Its main goal is to find a parking lot satisfying its requirements, stay parked for some amount of time, and eventually return home. To achieve this goal, the driver agent follows a sequence of plans that includes requesting parking information from area agents and selecting a suitable parking lot, navigating towards it, requesting entry to the corresponding parking agent, staying parked, and eventually leaving the parking lot (by requesting exit to the corresponding parking agent) and returning home.

The agent's decision-making is not based on a fixed sequence of actions. When a parking request fails, or a communication or movement operation is unsuccessful, **alternative plans** are triggered to retry the operation or switch to a different plan. Navigation similarly relies on rules for selecting suitable directions and handling obstacles or unsuccessful movements. This

allows the driver agent to **autonomously adapt** its behavior to the current situation.

Area Agent

The area agent is responsible for managing information about the parking lots within its area and providing this information to driver agents. Its knowledge is mainly represented by the **beliefs** it maintains about the parking lots in its area, including their positions and availability. Unlike the driver agent, the area agent does not directly perceive the environment. It acquires parking information through communication with the parking agents belonging to its area and updates its knowledge when availability changes. Its main behavior consists of collecting and maintaining parking information and responding to parking queries from driver agents. When a driver agent requests a suitable parking lot, the area agent uses its current knowledge to identify the closest parking lot to the driver agent satisfying the requested availability. If no suitable parking lot is known, the agent reports that no matching parking lot is available.

Parking Agent

The parking agent represents a parking lot and is responsible for managing its available spots and handling entry and exit requests from driver agents. Its **beliefs** include the parking lot's position and the number of currently available parking spots.

The parking agent primarily follows a **reactive behavior**. It responds to requests concerning parking information, entry, and exit. When its area agent requests its parking information, it responds with its position and its current number of available spots. When a driver agent sends an entry request, the agent checks whether a parking spot is available and grants or denies the request accordingly. A successful entry decreases the available capacity, while an exit increases it. These operations are handled **atomically** to ensure that concurrent requests do not lead to an inconsistent availability state. Finally, changes in parking availability trigger updates to the corresponding area agent, allowing the latter to maintain up-to-date information for future parking searches.

2.4 Inter-Agent Communication

Agents communicate with each other to acquire information, coordinate their behavior, and keep their local knowledge up to date. The communication

model mainly relies on two interaction patterns: **request-response** and **notification**. Request-response interactions are used when an agent requires information from another agent before proceeding, while notifications are used to communicate state changes without requiring an immediate response. In AgentSpeak, request-response interactions are performed through **askOne** queries, while notifications are performed through **achieve** sends. Communication outcomes can affect the agent’s subsequent reasoning: a successful response provides new information, a negative response may trigger an alternative plan, and a communication failure may cause the agent to retry the interaction or follow a fallback behavior.

The main communication interactions between the three agent types are described in the following sections.

2.4.1 Parking information collection

When an area agent starts, it collects information about the parking lots belonging to its area. As shown in Figure 2.3, using a *request-response* pattern, it requests the current information from each parking agent, which responds with the position and availability of the corresponding parking lot. The area agent then uses this information to build its local knowledge of the parking lots in its area. The parking agent therefore maintains the current state of its own parking lot, while the area agent maintains a local representation used for parking searches. In case of a communication failure, the request is retried indefinitely, since the area agent needs to obtain the parking information to build its local knowledge.

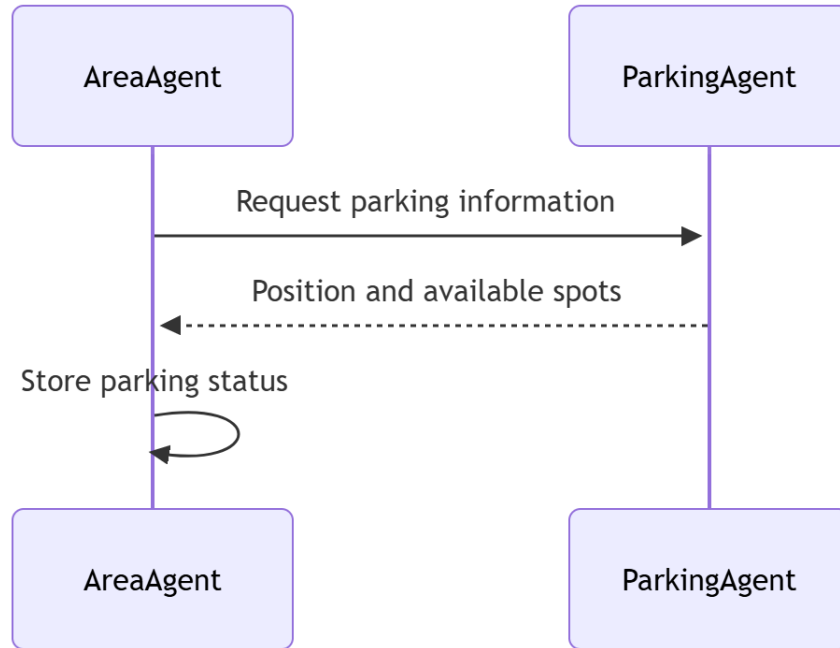


Figure 2.3: Sequence diagram illustrating the interaction between an area agent and a parking agent during the initial collection of parking information.

2.4.2 Parking search

Driver agents search for parking by communicating with area agents. As shown in Figure 2.4, a driver agent, using a *request-response* pattern, initially contacts the area agent responsible for the area in which it is located and specifies the minimum number of available spots required. The area agent uses its local knowledge to identify the closest parking lot satisfying the driver's requirement. If no suitable parking lot is available in the current area, the driver receives a negative response and contacts the area agent responsible for the nearest area. If communication with the current area agent fails, the driver agent retries the request **up to three times**. If the third attempt fails, the driver agent requests parking from the area agent responsible for the nearest area. If communication with the nearest area agent fails, or if the driver agent receives a negative response from it, the driver agent restarts the parking search protocol with the current area agent.

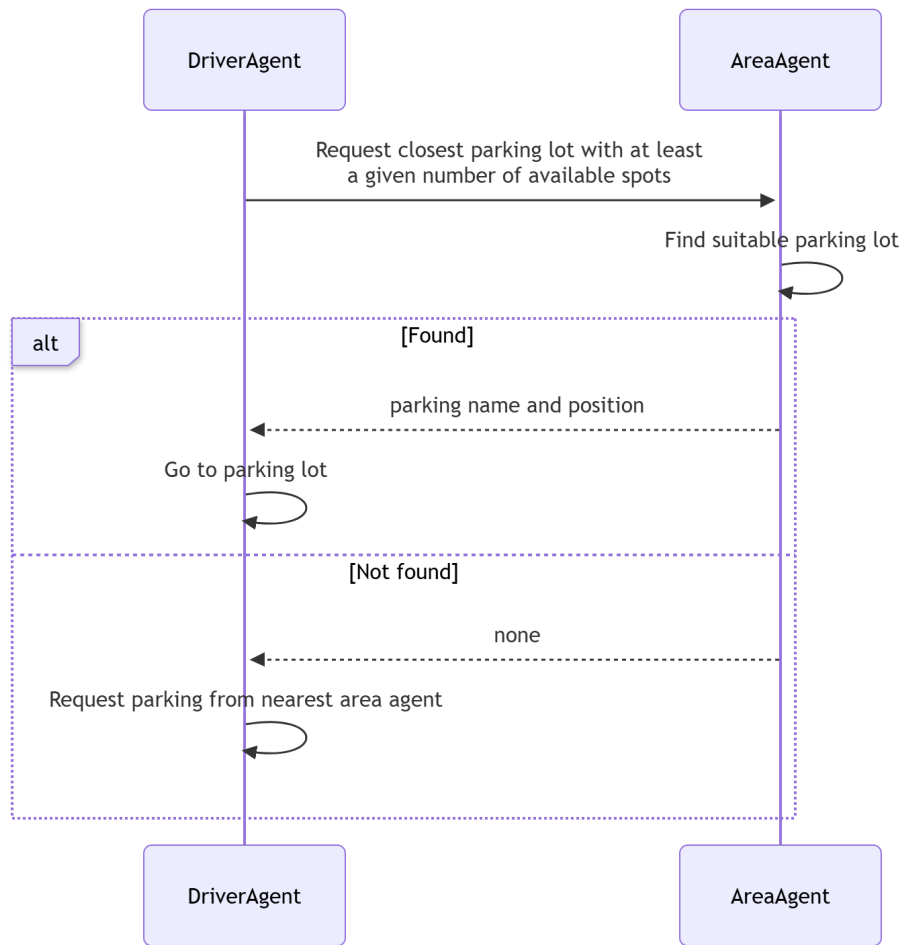


Figure 2.4: Sequence diagram illustrating the interaction between a driver agent and an area agent when requesting a parking lot.

2.4.3 Parking entry and exit

Once a driver agent reaches the selected parking lot, it communicates with the corresponding parking agent to request **entry**, as shown in Figure 2.5. The parking agent determines whether the operation can be performed according to the current availability of the parking lot. If at least one parking spot is available, the parking agent grants the entry request, performs the entry operation in the environment, and decreases the number of available spots. Otherwise, the request is denied, and the driver can search for another parking lot.

A similar behavior occurs when a driver wants to **exit** the parking lot.

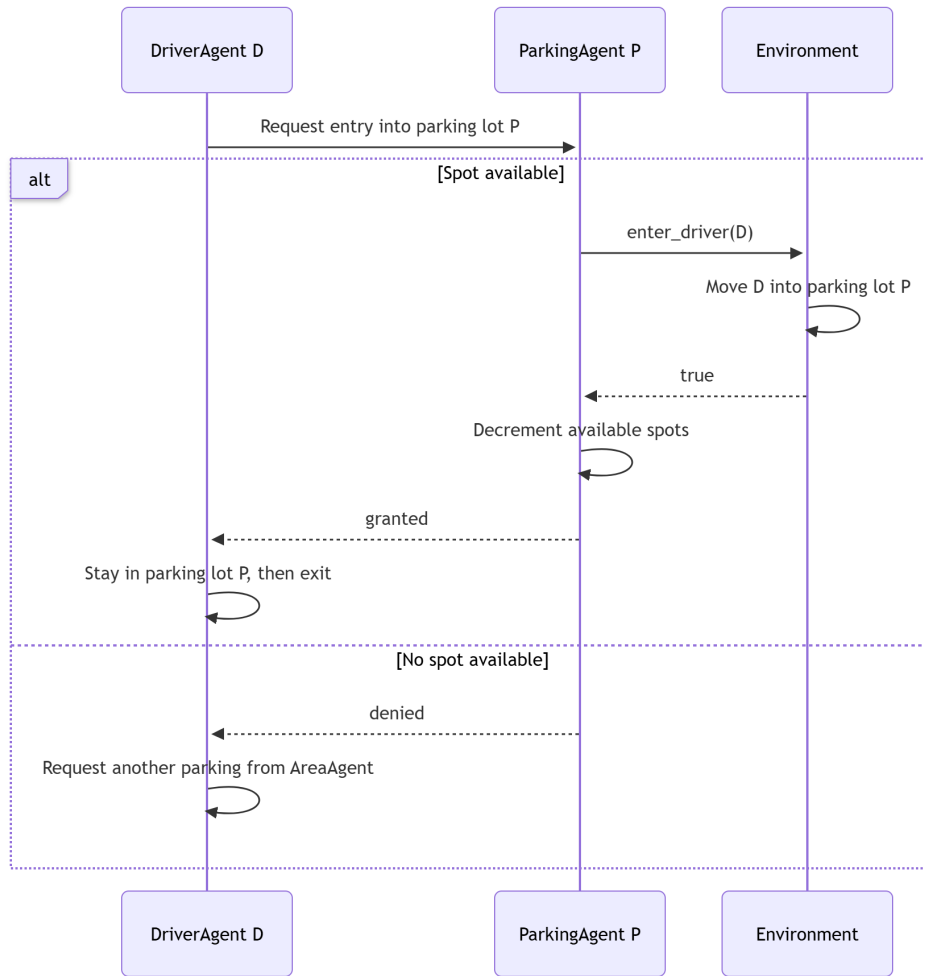


Figure 2.5: Sequence diagram illustrating the interaction between a driver agent and a parking agent when requesting to enter a parking lot.

In this case, the driver agent requests an exit from the parking agent, as shown in Figure 2.6. The parking agent performs the exit operation in the environment, increases the number of available spots, and confirms the exit to the driver agent, which then returns home.

In both scenarios, the interaction is *request-response* based. In case of a communication failure, the driver agent uses its environment percepts, in particular it checks its position, to determine whether the requested operation has actually been performed. If it has been performed, the driver agent can continue its behavior as if it had received a confirmation; otherwise, it can retry the communication. For entry, the driver agent retries the request up

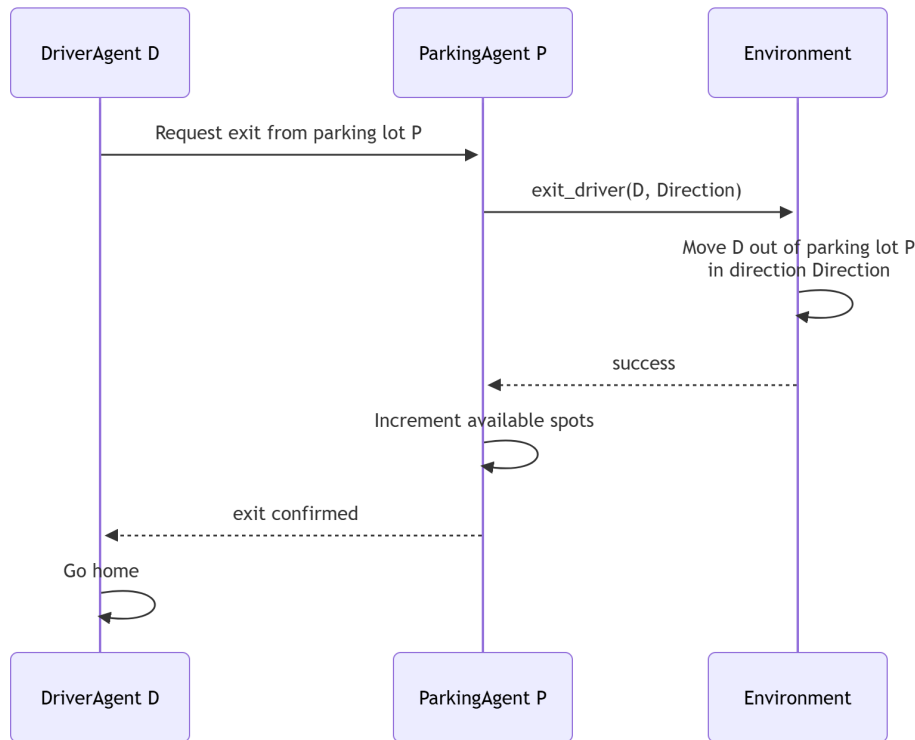


Figure 2.6: Sequence diagram illustrating the interaction between a driver agent and a parking agent when requesting to exit a parking lot.

to three times. If all attempts fail, it requests another parking lot from the current area agent. For exit, instead, the driver agent retries indefinitely because it is already inside the parking lot and must eventually complete the exit operation.

2.4.4 Parking availability updates

Whenever the availability of a parking lot changes, the corresponding parking agent communicates the new state to its area agent. As shown in Figure 2.7, the parking agent notifies the area agent of the availability update, upon which the area agent updates its local status of the parking lot accordingly. This communication is a *notification* rather than a request-response interaction because the parking agent does not require the area agent to return a result before continuing its operation. This update keeps the information used for parking searches up to date with changes occurring at the parking lot level. However, since the update is communicated between independent

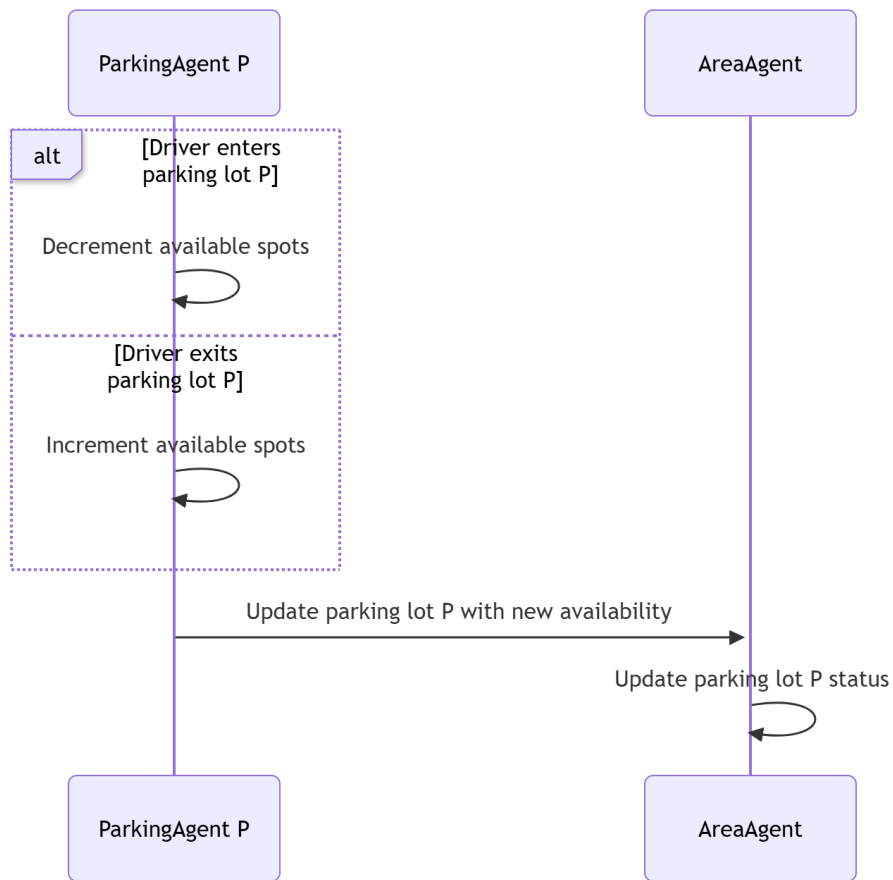


Figure 2.7: Sequence diagram illustrating the interaction between a parking agent and an area agent to update parking availability.

agents, the local information maintained by the area agent may temporarily differ from the authoritative state of the parking agent. This communication model therefore favors **local decision-making** and reduced communication during parking searches at the cost of potentially stale information.

Implementation

This chapter describes the implementation of the system, focusing on the concrete mechanisms used to realize the design presented in the previous chapter. It provides further details about the beliefs, rules, goals, and plans of each agent type, with each type implemented as a separate AgentSpeak program, as well as about the Java environment and its MVC components.

3.1 Environment

The `ParkingEnvironment` class is the interface between the agents and the City model. Through the `getPercepts` method, it provides the **percepts** to each agent according to its type as follows:

- Driver agents receive the percepts `position(X, Y)`, where `X` and `Y` are integers, representing their current position; `current_area(A)`, where `A` is the name of the current area; and, if it exists, `nearest_area(A)`, where `A` is the name of the nearest area to the driver other than the current one;
- Parking agents receive their position as `position(X, Y)`, where `X` and `Y` are integers;
- Area agents do not receive any percepts.

Instead, through the `executeAction` method, the `ParkingEnvironment` executes an action requested by an agent. The three possible **actions** are:

- `move(Direction)`: it can be performed by a driver agent and causes the driver agent to move one step in direction `Direction`;
- `enter_driver(Driver)`: it can be performed by a parking agent and causes the driver agent `Driver` to enter the corresponding parking lot;

- `exit_driver(Driver, Direction)`: it can be performed by a parking agent and causes the driver agent `Driver` to exit the corresponding parking lot in direction `Direction`.

The `executeAction` method therefore performs the requested action and returns its outcome, after updating the `CityView` and waiting a delay to show the simulation running.

Each percept is obtained from the `City`'s state, and each action modifies it. The initial configuration of the `City` is loaded from a JSON file, while its state is maintained in memory. The `City` is implemented in a **thread-safe** manner to support concurrent access.

3.1.1 GUI

The GUI is implemented using **Java Swing** and represents the view of the city, including the positions of drivers and parking lots. It is updated by the `ParkingEnvironment` after each agent action and is used exclusively to observe the execution of the MAS at runtime.

3.2 Agents

The following sections focus on the agent implementation aspects that are not directly visible from the high-level agent design.

3.2.1 Driver Agent

The driver agent is implemented through a set of plans for the different stages of the parking process, together with **rules** supporting its operations, particularly navigation and failure handling.

The **navigation mechanism** is implemented through a set of rules that derive preferred directions from the driver's current position and the target position. For a given pair of positions, the agent determines a primary direction and a secondary direction, together with their opposites. These directions define an ordered set of alternatives that can be tried during movement. Movement is therefore not implemented as a fixed sequence of actions. After each movement attempt, the agent re-evaluates its position and selects the next direction. If the preferred movement fails, alternative directions are considered. This allows the driver to react to obstacles or unsuccessful movements without requiring a precomputed route. This approach implements navigation as a **local decision-making mechanism** within the

driver agent. The agent determines its next action using its current percepts and the rules encoded in its own behavior, rather than relying on a centralized path-finding mechanism implemented in the environment. This design increases the agent's **autonomy**, as the environment only executes the requested movement and does not determine the route for the agent, allowing the agent to independently determine its behavior and adapt to the current situation.

The parking search and entry procedures use dedicated **failure-handling plans**. **Retry counters** are maintained as **beliefs** and are used to limit repeated attempts for operations such as parking requests and entry requests. When the maximum number of retries is reached, the corresponding fallback plan is triggered. For example, the parking request is sent as follows:

```
+!request_parking : current_area(A) & position(X, Y) &
  required_availability(R) <-
  .print("Requesting parking to area agent ", A);
  .send(A, askOne, available_parking(position(X, Y), R, P,
    Position), Answer, 3000);
  !process_parking_response(Answer).
```

Its timeout is handled as:

```
+!process_parking_response(timeout) : retries(R) &
  max_retries(M) & R < M <-
  .print("Area agent unavailable; retrying in 0.5s");
  .wait(500);
  +-retries(R + 1);
  !request_parking.
```

Finally, the fallback plan is:

```
+!process_parking_response(Error) <-
  .print("Couldn't contact area agent, max retries reached;
    contacting nearest area agent");
  +-retries(0);
  !request_parking_to_nearest_area_agent.
```

The driver agent also uses rules to determine whether it has reached the destination position as follows:

```
arrived(adjacent, position(X, Y), position(Xd, Yd)) :-
  adjacent(position(X, Y), position(Xd, Yd)).
arrived(exact, position(X, Y), position(Xd, Yd)) :-
  X == Xd & Y == Yd.
```

For parking, the driver agent considers the destination reached when it is adjacent to the parking lot. When returning home, instead, the destination is considered reached when the driver agent's position coincides with its home

position. These rules allow the same navigation plans to be **reused** for both cases as follows:

```

+!navigate_to(DestinationPosition, ArrivalCondition) :
    position(X, Y) &
    arrived(ArrivalCondition, position(X, Y),
            DestinationPosition) <-
    true.

```

3.2.2 Area Agent

The area agent implements its parking knowledge as **beliefs** of the form `parking_status(Parking, Position, Availability)`. Here, `Parking` identifies the parking agent, `Position` is the parking position and is represented as `position(X, Y)`, where `X` and `Y` are integers, and `Availability` is an integer representing the number of available spots in the parking lot. These beliefs are created in the `request_parking_info` plan during the initial collection of parking information, which is implemented as follows:

```

+!collect_all_parking_info <-
    .findall(P, parking(P), Ps);
    !collect_parking_info(Ps).

+!collect_parking_info([]).

+!collect_parking_info([P | Ps]) <-
    !!request_parking_info(P);
    !collect_parking_info(Ps).

```

The initial goal is `collect_all_parking_info`, and during its execution the area agent **spawns a new** `request_parking_info` **intention** for each parking lot in its area to collect its corresponding information. The `parking_status` beliefs are then updated when availability notifications are received.

The selection of a parking lot is implemented through the `closest_parking` rule as follows:

```

closest_parking(DriverPosition, RequiredAvailability, P,
    ParkingPosition) :-
    parking_status(P, ParkingPosition, Availability) &
    Availability >= RequiredAvailability &
    distance(DriverPosition, ParkingPosition, Distance) &
    not (
        parking_status(OtherP, OtherPosition,
            OtherAvailability) &
        OtherAvailability >= RequiredAvailability &

```

```

        distance(DriverPosition, OtherPosition, OtherDistance
        ) &
        OtherDistance < Distance
    ).

```

Given the driver agent's position and the required availability, the rule selects a parking lot that satisfies the requirement and for which no closer suitable parking lot exists, therefore finding the closest parking lot satisfying the availability requirement. The distance between the driver agent's position and the parking lot's position is calculated using the Manhattan distance. The area agent therefore performs parking selection **locally**, directly through rules over its own beliefs.

3.2.3 Parking Agent

The parking agent maintains the **current availability** of its parking lot as a **belief**. Entry and exit are implemented using **atomic plans**, ensuring that the availability check and the corresponding state update cannot be interleaved with other concurrent entry and exit operations. This is achieved by specifying the `@[atomic]` annotation before the corresponding test-goal plans. For example, the granted case for the entry operation is handled as follows:

```

@[atomic]
+?entry_status(granted)[source(D)] : available(A) & A > 0 <-
    .print("Processing entry request from driver ", D, ";
        available spots: ", A);
    enter_driver(D);
    .print("Entry granted for driver ", D, "; remaining spots
        : ", A - 1);
    -+available(A - 1).

```

The availability update sent to the area agent is triggered whenever the parking agent changes its availability as follows:

```

+available(Spots) : area(Agent) <-
    .my_name(P);
    .send(Agent, achieve, update_availability(P, Spots)).

```

The parking agent is consequently implemented mainly through reactive plans, particularly **test-goal plans** handling incoming requests, with the atomicity annotations providing the necessary protection for concurrent entry and exit operations.

Testing

The testing activity was divided into different levels to verify both the correctness of individual components and the behavior of the complete system. **Automated unit tests** were used to verify the functionality of the environment model, while **automated integration tests** were used to verify the interaction between the environment and the model. Finally, **manual end-to-end tests** were performed to evaluate the behavior of the complete multi-agent system, including agent reasoning, navigation, inter-agent communication, and failure handling.

Each class of the environment model is covered by **unit tests**. These tests verify the correctness of the main operations provided by the model, including driver movement, parking entry and exit, position management, and the handling of the city state. In particular, boundary conditions and invalid operations are also tested to ensure that the model does not reach an inconsistent state.

Integration tests are used to test the `ParkingEnvironment` in conjunction with the model. These tests verify the interface exposed to the agents by checking both the percepts returned by the environment and the effects of actions performed through it. In particular, they verify that the percepts reflect the state of the model and that agent actions produce the expected state changes.

Finally, **manual end-to-end tests** were performed on the complete multi-agent system to evaluate its behavior during realistic execution scenarios. These tests verify the complete interaction between driver, area, and parking agents, including parking search, navigation, entry, and exit. They also evaluate the agents' decision-making, including the area agents' selection of parking lots, the driver agents' adaptation of navigation to unsuccessful movements, and the activation of failure-handling plans.

Deployment

The deployment of the system is configured through two MAS2J files, which together define the environment, the agents, and their execution containers. As described in Section 2.2, the configuration distinguishes between the Main Container, which hosts the environment, configured by the `parkingMain.mas2j` file, and the additional JADE containers that host the agents, configured by the `parkingAgents.mas2j` file. The agent configuration in the second MAS2J file also specifies some initial beliefs for each agent, allowing different agent instances to be **configured with different initial beliefs**. For example, driver agents are initialized with different required parking availability values as follows:

```
d1 driver_agent[beliefs="required_availability(2)"] at "
  drivers";
d2 driver_agent[beliefs="required_availability(1)"] at "
  drivers";
```

Area agents and parking agents are configured with their respective parking relationships and initial availability as follows:

```
a1 area_agent[beliefs="parking(p1),parking(p2)"] at "area1";
p1 parking_agent[beliefs="area(a1),available(1)"] at "area1";
p2 parking_agent[beliefs="area(a1),available(2)"] at "area1";
```

The system is launched through two **Gradle tasks**. The first task starts the Main Container and initializes the simulated environment. The second task starts an additional JADE container and connects it to the existing Main Container. The latter task accepts the name of the container to be started, allowing the different agent containers to be launched using the same command. Therefore, the system can be started manually by executing the Gradle tasks for the Main Container and for each required agent container. Alternatively, the provided shell script `runParkingMAS.sh` automates the startup sequence by first launching the Main Container and then starting the containers for the two configured areas (`area1` and `area2`) and for the

driver agents (**drivers**). The Main Container is started before the other containers so that the JADE platform is available when they attempt to connect.

This deployment configuration allows additional area or driver containers to be started using the same mechanism, making it possible to extend the deployment without modifying the underlying agent implementation.

Future works

Several extensions could be considered to improve the current multi-agent system and to support more complex scenarios. In addition to improving the existing **failure-handling and fallback mechanisms**, the following extensions could be considered: the first two concern improvements to the existing agent behavior, while the third introduces a more complex environment.

A first possible improvement concerns the **local navigation mechanism** of driver agents. The current implementation uses a local decision-making algorithm that determines preferred directions from the driver's current position and target position, and adapts to unsuccessful movements by considering alternative directions. This mechanism could be extended while maintaining its local nature, for example by allowing the agent to take into account the history of previous movements and unsuccessful directions when making subsequent decisions. This would allow the agent to avoid repeatedly considering directions that have already proved ineffective, resulting in more adaptive local navigation without requiring a centralized path-finding mechanism.

A second improvement concerns the **knowledge maintained by area agents**. Their beliefs about parking availability are updated through notifications from parking agents and may therefore become temporarily outdated between updates. Future versions could explicitly represent the freshness of this information, for example by associating timestamps or validity information with parking availability beliefs. The area agent could then take the freshness of its knowledge into account when selecting a parking lot, or request updated information from a parking agent when necessary. This would provide a more explicit mechanism for reasoning about potentially stale knowledge.

Finally, the system could be extended with a more **complex environment model**. The current environment represents movement in a relatively simple spatial setting. A more realistic model could introduce roads, intersections, different movement constraints, and potentially blocked or unavailable

paths. This would provide a richer and more realistic environment in which agents could operate, while also creating additional challenges for the local navigation mechanism of driver agents.